Implementação de RAG no chatbot agronômico

Guia técnico das tecnologias e do fluxo de dados • 13 de outubro de 2026

1 O que estamos construindo

RAG significa Retrieval Augmented Generation, ou geração aumentada por recuperação. Antes de formular uma resposta, a aplicação procura informações em uma base externa e entrega os trechos relevantes ao modelo de linguagem. No projeto, essa base reúne publicações agronômicas e relatórios técnicos. A consulta à fazenda acrescenta dados estruturados de uma API.

O modelo não aprende permanentemente os PDFs quando eles são indexados. O projeto cria um índice pesquisável e monta o contexto de cada pergunta. Este relatório acompanha essa implementação em seis etapas; os exemplos de código são recortes didáticos, não substitutos dos arquivos completos.

Tecnologia	Função concreta no projeto
Python e Rich	Python executa a aplicação; Rich apresenta conversa, progresso e fontes no terminal.
pypdf	Extrai o texto das páginas dos PDFs para preparação da base.
LangChain	Oferece Document, divisão de texto, prompts e interfaces para modelos e busca.
Ollama	Executa localmente nomic-embed-text para vetores e llama3.2:3b para avaliação e geração.
Chroma	Persiste trechos, metadados e vetores na coleção embrapa_conhecimento.
LangGraph	Organiza etapas, estado e desvios: buscar, avaliar, reformular e responder.
Pydantic	Valida contratos de dados, incluindo a avaliação estruturada e dados da fazenda.
Node.js e JSON Server	Executam a API fictícia de fazendas, medições e planos de plantio.

Dois momentos diferentes

Preparação: arquivos → extração → trechos → embeddings → Chroma. Essa etapa acontece ao construir a base. Consulta: pergunta → busca → avaliação → contexto → resposta. Ela se repete durante a conversa. A API é consultada no momento da pergunta; as medições atuais não são indexadas no Chroma.

A aplicação existente é uma interface de terminal. LangChain conecta componentes e LangGraph controla a execução; nenhum deles é o modelo de linguagem. Os próximos passos detalham como esses componentes cooperam.

2 Transformar arquivos em trechos pesquisáveis

Extração e representação

Em src/ingest.py, DOC_METADATA_MAP define quais arquivos serão processados e associa título e cultura. Colocar um PDF na pasta, sozinho, não o inclui na ingestão. O código lê PDFs página a página com pypdf, limpa espaços e palavras hifenizadas e ignora páginas com menos de 50 caracteres úteis. Markdown é lido como texto.

O resultado é um Document do LangChain: page_content guarda o texto; metadata guarda sua origem. Essa estrutura permite que o conteúdo percorra extração, busca e resposta sem perder a referência. Exemplo ilustrativo de uma página:

```
from langchain_core.documents import Document

pagina = Document(
    page_content="Texto técnico extraído de uma página...",
    metadata={"source": "500perguntas-soja.pdf",
              "titulo": "Publicação sobre soja",
              "cultura": "soja", "pagina": 46}
)
```

Divisão em chunks

Um chunk é um trecho de texto usado como unidade de busca. Enviar um livro inteiro ao modelo encarece o processamento e pode exceder seu contexto. O RecursiveCharacterTextSplitter tenta dividir o texto usando separadores, como parágrafos, frases e espaços, preservando parte do trecho anterior na fronteira seguinte.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

divisor = RecursiveCharacterTextSplitter(
    chunk_size=750, chunk_overlap=120,
    separators=["\n\n", "\n", ". ", "? ", "! ", " ", ""]
)
trechos = divisor.split_documents([pagina])
```

Os valores 750 e 120 são os do projeto e representam caracteres, não tokens. A sobreposição busca preservar continuidade; o tamanho e a sobreposição efetivos dependem dos separadores. Cada trecho herda os metadados da página. Como a divisão parte de páginas separadas, ela não reconstrói automaticamente uma resposta que continua na página seguinte.

Entrada	Transformação	Saída
PDF ou Markdown	Extração e limpeza	Lista de Document com texto e origem.
Document de uma página	Divisão com sobreposição	Lista de trechos menores, ainda com metadados.

O que verificar antes de indexar

Confira se tabelas, números e unidades foram extraídos corretamente. O projeto não implementa OCR para páginas digitalizadas como imagem. O campo pagina indica a posição no PDF, que pode diferir do número impresso. Trechos sem contexto ou com extração ruim comprometem as respostas mesmo quando a busca funciona.

3 Criar embeddings e consultar o Chroma

Como o texto vira um vetor

Um embedding é uma lista de números calculada por um modelo para representar características do texto. Textos semanticamente relacionados tendem a ficar próximos nesse espaço. No projeto, OllamaEmbeddings chama nomic-embed-text pelo Ollama. Esse modelo produz vetores; ele não escreve a resposta da conversa [R3].

Exemplo conceitual: “temperatura para semear soja” → [0,12; −0,08; …]. Os números são fictícios, apenas para ilustrar o formato. Não representam temperaturas, probabilidades de acerto ou uma codificação diretamente legível por pessoas.

Persistência da base

```
from langchain_ollama import OllamaEmbeddings
from langchain_chroma import Chroma
from src.rag_config import CHROMA_DIR

embeddings = OllamaEmbeddings(model="nomic-embed-text")
base = Chroma(
    collection_name="embrapa_conhecimento",
    embedding_function=embeddings,
    persist_directory=str(CHROMA_DIR)
)
# Na preparação de uma base nova, com trechos já extraídos:
# base.add_documents(trechos)
```

A integração calcula os embeddings e o Chroma armazena vetores, textos e metadados. O diretório persistente permite reutilizar a base após encerrar o programa [R4]. src/rag_config.py resolve o caminho; ingestão e consulta precisam usar a mesma coleção e o mesmo espaço de embeddings. Trocar o modelo de embedding normalmente exige reconstruir o índice.

Recuperação por similaridade

```
buscador = base.as_retriever(
    search_kwargs={"k": 4, "filter": {"cultura": "soja"}}
)
documentos = buscador.invoke("Temperatura para semeadura da soja")
```

A pergunta também é convertida em vetor. O Chroma procura candidatos próximos, respeitando o filtro de cultura. k=4 solicita até quatro trechos, não quatro livros. A saída volta como lista de Document, com o texto original e seus metadados. src/rag_engine.py implementa essa operação em retrieve().

Por que recuperar não basta

Um trecho pode mencionar soja sem responder à pergunta. A similaridade ajuda a localizar assuntos; a etapa seguinte verifica a suficiência das evidências. Recuperação vazia também não prova que a informação não existe no acervo.

A ingestão atual retoma o trabalho pela contagem de trechos já gravados. Esse mecanismo não é uma atualização incremental confiável para arquivos alterados ou removidos. Não use o exemplo add_documents repetidamente sobre a base ativa sem uma política de IDs, versões e exclusão de registros antigos.

4 Montar o contexto e gerar a resposta

O papel do LangChain

No projeto, LangChain fornece componentes que recebem e entregam estruturas compatíveis: Document, ChatPromptTemplate, ChatOllama e StrOutputParser. O código conecta essas operações. A seleção do próximo passo fica no LangGraph; o texto da resposta é produzido pelo modelo llama3.2:3b executado pelo Ollama.

Para a geração, format_context() reúne os documentos aprovados e acrescenta identificadores de fonte, título, página e cultura. Assim, o prompt pode incluir algo como “FONTE [1] | PÁGINA 46 | CULTURA SOJA”, seguido do trecho. O modelo recebe a pergunta original junto dessas evidências.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_ollama import ChatOllama

prompt = ChatPromptTemplate.from_messages([
    ("system", "Responda com as evidências. Cite [1], [2]. "
               "Se forem insuficientes, informe a limitação."),
    ("human", "Pergunta: {pergunta}\nEvidências: {contexto}")
])
modelo = ChatOllama(model="llama3.2:3b", temperature=0.2,
                    num_ctx=4096, keep_alive="15m")
cadeia = prompt | modelo | StrOutputParser()
# cadeia.stream({"pergunta": pergunta, "contexto": contexto})
```

O símbolo | compõe uma sequência: o prompt organiza mensagens, o modelo gera uma saída e o parser extrai texto. stream() permite exibir fragmentos conforme chegam. O prompt real contém instruções adicionais sobre limites, fontes e tratamento dos documentos como dados. Este recorte é simplificado para estudo.

Tokens e limites da chamada

Tokens são unidades de texto do modelo: podem representar palavras, partes de palavras ou sinais. Não há equivalência fixa entre caracteres e tokens. num_ctx=4096 configura a janela que acomoda entrada e continuação; não reserva 4.096 tokens exclusivamente para a resposta.

Exemplo didático: 3.000 tokens de entrada e 400 de saída ocupam 3.400 tokens, deixando 696 de margem nessa janela. A entrada inclui instruções, pergunta, evidências e marcadores de mensagens. Cada nova avaliação ou reformulação processa sua própria entrada.

temperature=0,2 controla a amostragem, sem garantir correção factual. keep_alive="15m" mantém o modelo disponível por um período e reduz recarregamentos; não cria memória de conversa. O avaliador usa temperature=0 e num_predict=400. Esse limite de saída não está explicitamente aplicado ao gerador principal.

Saída esperada e limite

A resposta deve associar afirmações às evidências, por exemplo “O trecho [1] informa uma faixa de temperatura para semeadura”. Uma citação escrita pelo modelo não é, por si só, uma prova de correção; é necessário verificar se o trecho sustenta exatamente a afirmação.

5 Controlar a execução com LangGraph

LangGraph representa o processo como um grafo: nós executam funções, arestas definem a sequência e condições escolhem o caminho. Um estado compartilhado transporta pergunta, consulta, tentativas, documentos e resultado. StateGraph é montado e compilado em src/rag_graph.py [R1].

Fluxo documental implementado

```
INÍCIO → buscar → avaliar
                  ├─ suficiente → responder → FIM
                  ├─ ambígua → esclarecer → FIM
                  └─ insuficiente
                      ├─ primeira busca → reformular → buscar
                      └─ segunda busca → sem_evidencia → FIM
```

buscar chama retrieve(); avaliar chama evaluate(); reformular chama rewrite_query(); responder chama generate(). O limite MAX_SEARCHES=2 permite uma nova busca. A pergunta original permanece no estado enquanto search_query pode mudar. Isso evita substituir a intenção do usuário pela reformulação.

Decisão estruturada com Pydantic

O avaliador pede ao modelo uma saída conforme CompactAssessment. Exemplo ilustrativo, usando um trecho didático que realmente deveria estar entre os candidatos:

```
{
  "decision": "sufficient",
  "evidence_quotes": [
    {"source_id": 1, "text": "A ficha está no capítulo três."}
  ],
  "clarification": ""
}
```

Pydantic verifica tipos, campos e limites. A aplicação confere se cada passagem aparece na fonte indicada, normalizando espaços, caixa, acentos e hifenização. Só então deriva os IDs das evidências aprovadas. O contrato admite no máximo duas passagens; formato válido e correspondência textual ainda não comprovam relevância agronômica.

Situação	Comportamento do projeto
Informação suficiente	Encaminha documentos aprovados ao gerador.
Pergunta incompleta	Solicita esclarecimento; o usuário deve enviar uma pergunta completa.
Duas buscas insuficientes	Encerra sem inventar uma resposta documental.
Avaliação inválida	Interrompe a geração com mensagem de falha de validação.

Estado não é memória persistente

Cada pergunta começa com um estado novo. Não há checkpoint ou histórico persistido entre mensagens. O uso de LangGraph permite desenvolver esses recursos, mas não os ativa automaticamente. O tutorial oficial de RAG com LangGraph ajuda a estudar a arquitetura; o projeto possui seu próprio roteamento e validações [R2].

6 Integrar a API e executar o fluxo completo

Como os dados atuais entram

Node.js executa api/server.cjs com JSON Server, expondo os dados de api/db.json. src/weather_api.py consulta a API por HTTP e valida identidade e atualidade. src/planting_context.py trata o plano. A seed renova a simulação; não coleta sensores e não exige reindexar os documentos.

O grafo externo, em src/assistant_graph.py, escolhe a rota por regras de texto: uma pergunta documental usa RAG; uma pergunta sobre a medição consulta a API; uma operação agrícola dependente do clima combina fontes. Consultas diretas podem terminar sem geração por LLM.

Exemplo de uma pergunta combinada

Entrada: “Quais as condições climáticas hoje? Posso plantar soja?”. Primeiro, o sistema consulta a última medição e o plano correspondente. No cenário fictício: ar 28 °C, solo 25 °C, umidade volumétrica 28%, solo não encharcado e data dentro da janela cadastrada.

Se houver dados vencidos, ausentes ou restrições do plano, a análise é interrompida. Se as verificações passarem, a busca documental consulta requisitos de temperatura para semeadura de soja. O grafo avalia os trechos e encaminha evidências aprovadas para a comparação.

Quando reconhece a frase de faixa térmica prevista pelo código, Python extrai os limites e compara os números diretamente. Portanto, nem toda resposta vem de texto gerado pelo modelo. Em outros casos, generate_with_weather() fornece documentos e dados da API ao LLM.

Saída interpretada: “O cenário atende às regras fictícias do plano; as temperaturas estão numericamente no intervalo recuperado”. Isso não autoriza plantio real. O trecho pode não distinguir ar de solo; limites de umidade e janela da API são simulados. Decisões reais exigem dados medidos, critérios aplicáveis e validação agronômica.

Como acompanhar no projeto

Leia src/ingest.py → src/rag_config.py → src/rag_engine.py → src/rag_graph.py → src/assistant_graph.py. Complete com weather_api.py, planting_context.py e src/chatbot_cli.py. O primeiro prepara a base; os demais conectam busca, modelo, regras, API e interface.

```
# No PowerShell, na pasta chatbot, em terminais separados:
.\iniciar_api.bat
.\iniciar_chatbot.bat
```

Para usar o RAG, Ollama precisa estar disponível com os dois modelos, e a base Chroma deve estar preparada. Compare três perguntas: “Temperatura para semear soja?”, “Temperatura da fazenda hoje?” e a pergunta combinada acima. Observe fontes e percurso, não apenas o texto final.

Referências para estudo

[R1] LangGraph overview

[R2] Build a custom RAG agent with LangGraph

[R3] Ollama Generate embeddings

[R4] Chroma Introduction

Base principal: código local do projeto, consultado em 13/10/2026. Os exemplos explicam a implementação existente; não representam uma nova versão executável do chatbot.